

對話式 ERP 架構設計 (繁體中文)

AI 取代教育訓練 + 自然語言全 CRUD 操作 erpilot 的核心差異化定義 — 這份文件 = 北極星

版本：v1.0 (2026-05-15) · 狀態：Phase 1 動工前最終版



目錄

- [1. 願景：為何重寫產品 DNA](#)
- [2. 現狀對照：1.5 / 8](#)
- [3. Naive 設計的 6 個致命災難](#)
- [4. 6 層架構 \(端到端流程 \)](#)
- [5. 核心設計原則 \(7 個 \)](#)
- [6. 4 階段 Roadmap](#)
- [7. 成功指標 KPIs](#)
- [8. 風險與緩解](#)
- [9. 代碼層級對應](#)

1. 願景

員工不需要知道 ERP 有多少功能表、按鈕、欄位。他想做什麼，就講；AI 把語言翻譯成系統動作。教育訓練 = 教 AI 助手怎麼用，而不是教 ERP 介面。

這是徹底翻轉「軟體設計傳統」—— 從「為操作員設計 UI」變成「為對話設計系統」。SAP / Odoo / Workday 都做不到。我們敢做這個是真正的差異化。

1.1 對應到客戶感受

傳統 ERP	erpilot 對話式
員工受訓 1-3 個月才會用	2 小時上手：知道怎麼跟 AI 說話即可
找功能要記得選單第 N 層	「我想做 X」，AI 帶您去（或直接做）
看不懂專業術語要查手冊	「什麼是 PO 三方比對？」AI 用白話解釋
操作錯了改不回來	「取消剛才那筆」5 分鐘內可 undo
報表月底才有	「今天狀況」隨時問、隨時得

2. 現狀對照

能力	UI (12 個頁面)	自然語言 (26 個 AI tool)
查詢 Read	✓ 有 list 頁	✓ 26 個 tool 全是查的
新增 Create	✓ 有 form	✗ 0 個 tool
修改 Update	✗ 沒做	✗ 0 個 tool
刪除 Delete	✗ 沒做	✗ 0 個 tool

完成度 = 1.5 / 8 = 19%

這份文件目的：把另外 81% 補完，且補對方向（不只是加 tool，要重新設計對話流）。

3. Naive 設計的 6 個致命災難

若我們直接寫 `delete_customer` / `update_inventory` 26 個寫入 tool 丟給 AI 用，會出 6 件災難。這 6 件事決定了我們的架構選擇：

#	災難	範例	解法 (在第 5 節)
1	幻覺 Hallucination	AI 編 <code>part_no = "M6-NEW"</code> 但 DB 沒有	原則 #1 Risk Tier + 原則 #4 Disambiguation
2	歧義 Ambiguity	「改一下那筆訂單」—— 哪筆？	原則 #4 Disambiguation
3	無確認 No Confirm	「刪掉客戶 A」AI 立刻刪	原則 #2 Confirmation Card
4	沒 undo	刪了拉不回	原則 #5 Undo Token
5	不會問 Missing Slots	「給中鋼下 PO」—— 數量？單價？AI 自編	原則 #3 Slot-filling
6	越權 Privilege Escalation	業務透過 AI 下單 100 萬 (本來無此權限)	原則 #7 RBAC × AI 整合

4.6 層架構

每一層解決一個 naive 災難。這是端到端流程：

使用者：「幫我訂 1000 個 M6 螺絲給中鋼」

↓

Layer 1 Intent + Slot Extraction

IntentClassifier → Agent = PurchaseAgent

Slots: { part: "M6", qty: 1000, supplier: "中鋼" }

↓ 缺 price, 自動 fetch 上次價 (last_supplier_price)

解決：災難 #5 (missing slots 反問)

↓

Layer 2 Disambiguation

"M6" 對應 3 個 part: M6-BOLT-20 / M6-NUT / M6-WASHER

AI: "您是指 M6-BOLT-20? 還是另兩個?"

解決：災難 #2 (ambiguity)

↓

Layer 3 Risk Classification + RBAC

tool = create_purchase_order → 中風險

金額 = 1000 × \$0.5 = \$500 < \$10K threshold

user.has("purchase.order.create") ? √

→ Tier 2 (confirm card 必須)

解決：災難 #1 (hallucination via real-data check)

災難 #6 (RBAC × AI 整合)

↓

Layer 4 Confirmation Card

AI 回 JSON:

```
{
  type: "confirm_required",
  summary: "建立 PO 給中鋼 · M6-BOLT-20 × 1000 · NT$500",
  action: "create_purchase_order",
  args: {...},
  buttons: ["✓ 確認", "← 修改", "X 取消"]
}
```

前端渲染為 inline card, 使用者點按鈕

解決：災難 #3 (無確認)

↓ 老闆按「✓ 確認」

Layer 5 Execute + Audit + Undo Token

create_purchase_order(confirmed=True, undo_token=xxx)

→ PO-2026-105 已建立

DecisionLog 寫入：誰、何時、AI 推理過程、結果

Undo token 有效 5 分鐘 → 寫 redo_stack

解決：災難 #4 (沒 undo) + 全程可稽核

↓

Layer 6 Conversational Memory

AI: "已建立 PO-2026-105。預計 7 天到貨。"

若要取消, 5 分鐘內說『取消剛才那筆』即可。"

```
Session 記住 "剛才那筆" = PO-2026-105
跨 session 記住老闆常問的問題 / 公司專用術語
解決：對話「真的像對話」而非每句獨立
```

5. 核心設計原則

原則 #1 : Tool Risk-Tier 三分類

每個 tool 在註冊時必須帶 `risk_tier` :

Tier	範例	行為
read	list_parts / query_inventory	直接執行、不需確認
soft-write	create_draft_po / save_search_filter	直接執行 + 提供 easy undo
hard-write	approve_po / delete_customer / post_journal	必須回 Confirmation Card

原則 #2 : Confirmation Card 模式

Hard-write tool 的 AI 不直接執行。它回 JSON 給前端：

```
{
  "type": "confirm_required",
  "summary_zh": "建立 PO-105 給中鋼 · M6-BOLT-20 x 1000 · NT$500",
  "summary_en": "Create PO-105 to ZhongGang · M6-BOLT-20 x 1000 · NT$500",
  "action": "create_purchase_order",
  "args": { "supplier_id": "S-001", "items": [...] },
  "undo_eligible": true,
  "buttons": ["confirm", "adjust", "cancel"],
  "expires_at": "2026-05-15T10:30:00Z"
}
```

前端渲染成 inline card，使用者點 confirm → 後端用 `args` 真的 call tool。

原則 #3 : Slot-filling 對話機

每個 hard-write tool 有 `required_slots` 清單。缺欄位時 AI 反問：

```

User: "給中鋼下 PO"
AI: "M6-BOLT-20? M6-NUT? 哪個?" (disambiguation)
User: "M6-BOLT-20"
AI: "數量多少?" (missing slot: qty)
User: "1000 個"
AI: "好的, 建單 1000 個 M6-BOLT-20 給中鋼公司, 單價 NT$0.5 (上次成交價) 總額 NT$500。確認嗎?"

```

實作：Tool registry 定義 `slots`，AI 在 LLM prompt 中用 ChainOfThought 找缺欄位。

原則 #4：Disambiguation 流程

當 entity-resolver 對單一名稱找到 >1 筆 candidate：

```

# In purchase_tools.py
async def _create_po(db, user, supplier: str, items: list[dict]):
    candidates = await search_supplier(db, supplier)
    if len(candidates) > 1:
        return {
            "type": "disambiguation",
            "candidates": [{"id": s.id, "label": s.name} for s in candidates],
            "prompt": f"「{supplier}」對應到 {len(candidates)} 家供應商，是哪一個?",
        }
    ...

```

原則 #5：Undo Token 機制

每個 soft/hard-write tool 執行後寫一筆 `ActionHistory`：

```

{
  "id": "ah-uuid",
  "session_id": "...",
  "user_id": "...",
  "tool": "create_purchase_order",
  "args_before": {}, # 為了 undo, 存 forward args
  "args_after": {...}, # 執行後狀態
  "undo_recipe": "delete_purchase_order(po_id=...)",
  "executed_at": "...",
  "expires_at": "2026-05-15T10:35:00Z",
  "undone": false
}

```

User 說「取消剛才那筆」→ AI 找 session 最後一筆 ActionHistory + 未過期 → 跑 undo_recipe。

原則 #6：教育替代三件套

glossary tool (術語白話化)：

User: "什麼是 PO 三方比對?"

AI: "三方比對 = PO 訂單 vs 供應商出貨單 vs 我們收料單，三邊數量金額對得起來才能付款。我們系統會自動比對，異常時推播給您。"

workflow guide tool (流程指引) :

User: "月結要怎麼做?"

AI: "月結 7 步：① 凍結昨日交易 ② 跑 AR aging ③ 跑 AP aging ④ 跑庫存 valuation ⑤ 結算損益 ⑥ 主管 review ⑦ Close month。要從第 1 步開始嗎?"

learn-our-term tool (學公司專用語) :

User: "把 M6 BOLT 20mm 我們叫『梅花小』"

AI: "好的，已記住您公司術語：『梅花小』 = M6-BOLT-20。下次說『梅花小庫存』我會直接懂。" 寫到 TermAlias 表。

原則 #7 : RBAC × AI 整合

每個 tool 註冊時帶 `required_permission` :

```
@register_tool(
    domain="purchase",
    risk_tier="hard-write",
    required_permission="purchase.order.create",
    slots=["supplier", "items"],
)
async def create_purchase_order(...): ...
```

AI 在 Layer 3 檢查： `user.has(tool.required_permission)`，沒權限就拒絕：「您沒有建 PO 的權限，請洽主管」。

6.4 階段 Roadmap

Phase 1 (Week 1) — Foundation

目標：對話式架構基建跑通，第一個 hard-write tool e2e 可示範

詳見 `CONVERSATIONAL_ERP_PHASE1_SPEC_ZH.md`。

Day	工作	交付
1	Tool registry + risk-tier 框架	<code>app/agents/registry.py</code>
2	<code>ConfirmCardResponse</code> schema + frontend ConfirmCard 元件	schema + React component
3	第一個 hard-write tool : <code>create_purchase_order_with_confirm</code>	tool + 對話端到端
4	Slot-filling 缺欄位反問機制	LLM prompt strategy
5	E2E demo : 「幫我訂 1000 個 M6」全流程 + 錄影	demo video

Phase 2 (Week 2) — 16 個核心 write tool

Domain	Write Tools
Inventory	adjust_stock / create_part / update_safety_stock
Purchase	create_po / approve_po / cancel_po
Sales	create_so / confirm_so / ship_so
Production	create_wo / release_wo / complete_wo
Quality	create_capa / mark_ncr_resolved
Accounting	post_journal / close_month

每 tool 半天 (risk-tier 框架已建好 → 後續 tool 是線性投入) 。

Phase 3 (Week 3) — 對話智慧

- Disambiguation 主動發問
- Glossary tool (公司 + 通用 ERP 名詞 200 條)
- Workflow guide tool (10 個典型工作流：月結 / 盤點 / 退貨 / ...)
- Undo / rollback：5 分鐘可撤回
- Session memory：「剛才那筆」、「上次提到的」
- TermAlias 個人化術語學習

Phase 4 (Week 4) — 規模化 + 上線

- 個人化 (老闆早報 / 業務 quick-reply)

- 公司術語學習 (一次教永久記)
- Mobile 對話介面整合
- Frontend 12 頁加 Edit/Delete 按鈕 (給不想對話的 power user 保底)
- Phase 1-3 整套 e2e demo

7. 成功指標

MVP 指標 (4 週後該看到的)

指標	目標	怎麼量
自然語言成功率	「使用者意圖 → AI 正確執行」 ≥ 85%	內部 100 條 golden query 回歸測試
平均教育訓練時數	< 2 小時 / 員工	demo 給 5 個非技術員工，計時上手
CRUD 覆蓋率	全 12 domain × CRUD = 48 case 都能跑	端到端測試矩陣
誤操作攔截率	hard-write 100% 經 confirm card	全 e2e 測試
客戶上線教育時間	< 1 天	新客戶 onboarding 時長

長期指標 (v2.0 之後)

指標	目標
AI 操作佔總操作比例	> 60%
使用者請求平均耗時	< 30 秒 (含確認)
主動推播 / 客戶問之比	> 3:1 (AI 比客戶先想到)

8. 風險與緩解

風險	嚴重度	緩解
LLM 幻覺造成資料毀損	● 致命	Confirmation Card + Risk-tier + Undo 三道防線
LLM 成本爆炸	● 高	三層智能路由 (rule 40% + ollama 50% + cloud 10%) + rate limit
使用者抗拒「要先確認」(嫌慢)	● 中	Tier 設計：read 直接執行、soft-write 也直接執行、僅 hard-write 確認
Prompt injection 越權	● 致命	既有 Prompt Safety 層 + RBAC × AI 雙重檢查
對話狀態混亂 (多輪打架)	● 中	Session 30 min TTL + 顯式「reset」指令
多語言混雜 (中夾英)	● 低	LLM 本身支援

9. 代碼層級對應

新增檔案

```

backend/app/agents/
├─ registry.py           NEW · Tool risk-tier 註冊框架
├─ confirm_card.py       NEW · ConfirmCardResponse schema
├─ slot_filling.py       NEW · 缺欄位反問策略
├─ disambiguation.py     NEW · 歧義解析
├─ action_history.py     NEW · undo / rollback
└─ domains/
    ├─ purchase_write_tools.py  NEW · Phase 2 寫入 tool
    ├─ sales_write_tools.py     NEW
    └─ ...

backend/app/models/
├─ action_history.py     NEW · ORM model for undo
└─ term_alias.py        NEW · 公司術語學習

frontend-desktop/src/components/
├─ ConfirmCard.tsx       NEW · 確認卡片元件
├─ DisambiguationCard.tsx NEW · 多選卡片元件
└─ ChatMessage.tsx      MODIFIED · 支援 inline card

frontend-desktop/src/pages/Chat.tsx  MODIFIED · 對話狀態機

docs/
├─ CONVERSATIONAL_ERP_DESIGN_ZH.md (本文件)
├─ CONVERSATIONAL_ERP_DESIGN_EN.md
├─ CONVERSATIONAL_ERP_PHASE1_SPEC_ZH.md
└─ CONVERSATIONAL_ERP_PHASE1_SPEC_EN.md

```

修改檔案

- `app/api/chat.py` — 加 ConfirmCard 處理路徑
- `app/agents/engine.py` — 引入 risk-tier gating
- `app/core/security.py` — RBAC × AI 整合
- `app/models/ai_governance.py` — DecisionLog 加 confirm_token / undo_token



相關文件

- [Phase 1 Day-1 to Day-5 Spec](#)
- [Agent Catalog \(現有 26 tool \)](#)
- [系統架構藍圖](#)
- 英文版： `CONVERSATIONAL_ERP_DESIGN_EN.md`

版本：v1.0 · 最後更新：2026-05-15 · 狀態：Phase 1 動工前最終版